PORA ACADEMY

Data Science

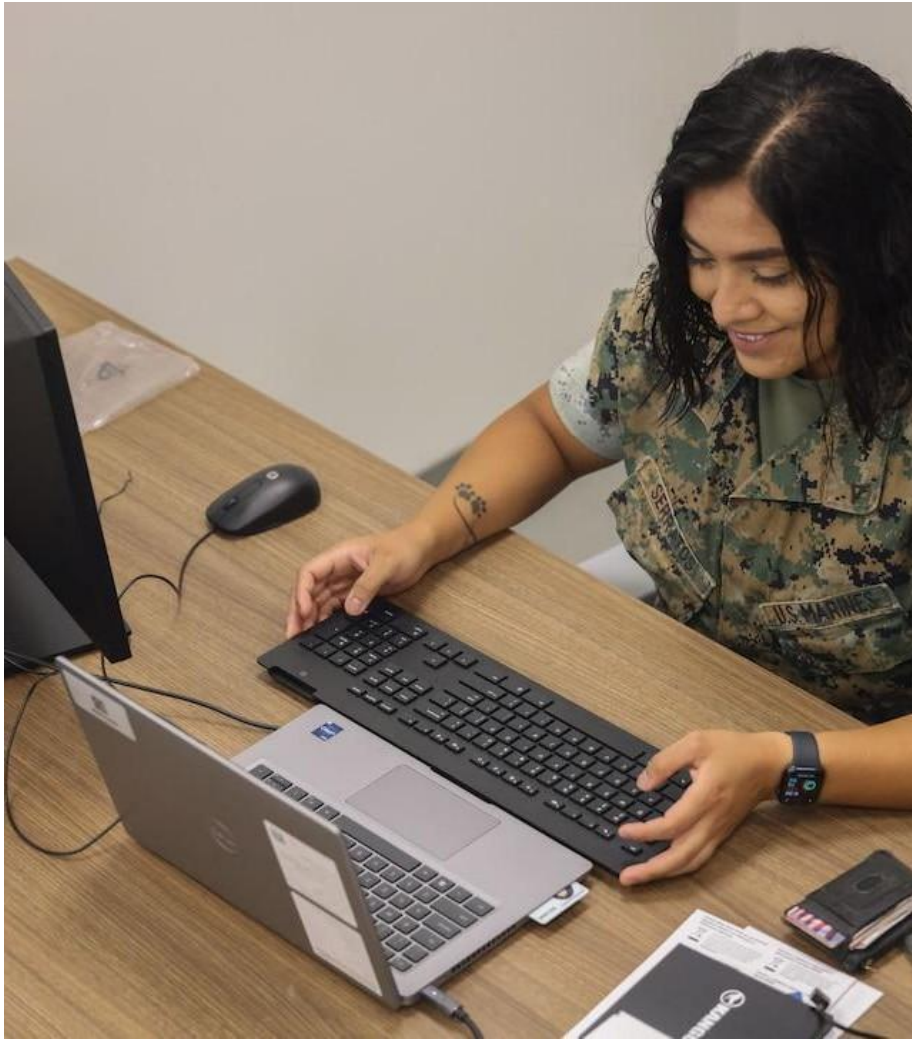
Python Basics to Data Science

Why use Python?

- Simple syntax and readability
- Extensive libraries
- Large community of support
- Often preferred for deploying machine learning models in production environments.
- Widely adopted in the technology industry

Software and Setup

- Jupyter Notebook
  - Download and Install Anaconda https://www.anaconda.com/download
  - From Anaconda launch Jupyter Notebook
- Google Colab



Data Types

- Integers (int): Whole numbers (e.g., 5, -3)
- Floating Point (float): Decimal numbers (e.g., 3.14, -2.7)
- Strings (str): Sequence of characters (e.g., "Hello, World!")
- Booleans (bool): Represents True or False values
- Lists, Tuples, and Dictionaries: Collections used to store multiple items.

N.B: Understanding data types is crucial for effective programming and avoiding type-related errors.

Variables

- **Meaning**: A variable is a named location in memory that stores data and can be manipulated during program execution.
- Variable Assignment

  Syntax: `variable_name = 'value'`

- Local Variables: Defined within a function and accessible only inside that function.
- Global Variables: Defined outside of all functions and accessible throughout the program.

Variables: Naming Convention

- Use descriptive names for a variable to make code more readable.
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alphanumeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

Expressions

- **Meaning:** An expression is a combination of variables, operators, and values that produce a result.

- **Types of Expressions:**
  - Arithmetic Expressions: Perform mathematical operations (e.g. 3 + 5 * 2)
  - String Expressions: Concatenate or manipulate (e.g. strings "Hello, " + "World!")
  - Boolean Expressions: Evaluate to True or False (e.g. 5 > 3)

N.B: Expressions are evaluated according to the precedence of operators (e.g., multiplication before addition

Numerical Operations

- Arithmetic Operators:
  - Addition (+)
  - Subtraction (-)
  - Multiplication (*)
  - Division (/)
- Examples:
  - a + b, x * y
- Advanced Operators
  - Exponentiation (**): 2 ** 3 results in 8

Logical Operations

- Comparison Operators:
  - Equal (==)
  - Not Equal (!=)
  - Greater Than (>)
  - Less Than (<)
- Logical Operators:
  - AND (and)
  - OR (or)
  - NOT (not)
- Examples: a > b and b < c, not is_empty

Working with Strings

- Concatenation: Combine strings using + (e.g., "Hello" + " World")
- Slicing: Extract substrings using [start:end] (e.g., text[0:5])
- Repetition: Repeat strings using * (e.g., "Hi" * 3 gives "HiHiHi")
- Indexing: Access individual characters using text[index]

  Transformations:Uppercase/Lowercase: Convert case with upper() or lower()
- Trimming: Remove whitespace with strip(), lstrip(), or rstrip()
- Replacing: Replace characters or substrings with replace(old, new)

String Formatting

- Formatted Strings:
  - Old Style (%): "Hello, %s!" % name
  - New Style (str.format()): "Hello, {}!".format(name)
- f-Strings (Python 3.6+): f"Hello, {name}!"
- Advantages:
  - Easily insert variables into strings
  - Control the display of numbers (e.g., {:.2f} for two decimal places)
- Format strings for readability and clarity in outputs

Lists

A list is an ordered, mutable collection of items.

- **Characteristics:**
  - Can contain multiple data types.
  - Supports indexing and slicing.
  - Allows modification (adding/removing elements).

Common Operations

```
my_list = [1, 2, 3, "apple", "banana"]
my_list.append("cherry")  # Add an element
my_list.remove(2)  # Remove an element
for item in my_list:
    print(item)  # Iterate through list
```

Tuples

A tuple is an ordered, immutable collection of items.

Differences from Lists:

- Cannot be modified after creation.
- More memory efficient than lists.

Common Operations

```
my_tuple = (1, 2, 3, "apple", "banana")
print(my_tuple[0])  # Accessing elements
def process_tuple(t):
    return t[::-1]  # Returns reversed tuple
print(process_tuple(my_tuple))
```

Dictionaries

A dictionary stores data as key-value pairs.

Characteristics:

- Keys must be unique and immutable.
- Values can be of any data type.

Common Operations

```
my_dict = {"name": "Alice", "age": 25}
my_dict["city"] = "New York"  # Adding a key-value pair
del my_dict["age"]  # Removing a key
for key, value in my_dict.items():
    print(f"{key}: {value}")
```

Conditions

Conditional statements control the flow of execution based on conditions.

- Common Structures:
  - if, elif, else

Common Operations

```
age = 18
if age < 18:
    print("Minor")
elif age == 18:
    print("Just turned adult")
else:
    print("Adult")
```

Loops

Loops allow iteration over sequences or repeated execution of code.

Types:

- `for` loop: Iterates over sequences.
- `while` loop: Executes while a condition is true.

Common Operations

```
for i in range(5):
    print(i)  # Looping through numbers

count = 0
while count < 3:
    print("Counting:", count)
    count += 1
```

Functions

A function is a reusable block of code designed to perform a specific task.

The process of creating a function using the **def** keyword.

Characteristics:

- Helps in code reusability.
- Can accept parameters.
- Can return values.

Common Operations

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Alice"))
```

Advanced Features

- Default parameters.
- Multiple return values.

Common Operations

```
def user_info(name, age=30):
    return f"Name: {name}, Age: {age}"

print(user_info("Bob"))  # Uses default age
print(user_info("Charlie", 25))  # Custom age
```

Conclusion

Key takeaway

- Python provides versatile data structures for different needs—lists, tuples, and dictionaries each have unique properties. Lists allow modification, tuples offer immutability, and dictionaries enable efficient key-value storage.

- Control flow structures (loops, conditions, and functions) Enhance code efficiency—loops automate repetitive tasks, conditional statements guide decision-making, and functions improve reusability and modularity.

Questions and Answers session

PORA
ACADEMY

THANK YOU!